[Institution name to be added]

Department of Computer Science

Project Report

Hybrid Cloud and Distributed Ledger E-Voting System

A Blockchain-Based Secure Electronic Voting Platform using Hedera Consensus Service

Submitted by: Ved Yadav
Student ID: [Student ID to be added]
Supervisor: [Supervisor name to be added]
Degree: Bachelor of Technology / Master of Technology

October 2026

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology / Master of Technology

Declaration

I, Ved Yadav, hereby declare that the work presented in this report entitled **Hybrid Cloud and Distributed Ledger E-Voting System** is my own original work carried out under the supervision of [Supervisor name to be added]. This work has not been submitted for any other degree or diploma at this or any other institution.

All sources used have been acknowledged and referenced appropriately. Any material from other sources has been properly cited.

Signature: _____
Name: Ved Yadav
Student ID: [Student ID to be added]
Date: October 29, 2026

Certificate

This is to certify that the project work entitled **Hybrid Cloud and Distributed Ledger E-Voting System** submitted by Ved Yadav(Student ID: [Student ID to be added]) is a bonafide work carried out by them under my supervision and guidance. This work has not been submitted elsewhere for the award of any other degree.

Signature: _____

Name: [Supervisor name to be added]

Designation: Supervisor

Date: October 29, 2026

Abstract

Traditional electronic voting systems suffer from fundamental trust issues: centralized databases can be manipulated by administrators, audit trails are often incomplete, and voters have no independent mechanism to verify that their votes were recorded correctly. This project addresses these challenges by designing and implementing a hybrid cloud and distributed ledger e-voting system that leverages the Hedera Consensus Service (HCS) for cryptographic immutability and transparent auditability.

The system adopts a Web2.5 architecture that decouples identity management from vote consensus. In the implemented prototype, authentication and authorization are handled by a Node.js/Express backend with JWTs and MongoDB, while vote recording occurs on the Hedera testnet through an HCS topic. The backend prevents duplicate participation with an atomic voter-state update and rolls that update back when the HCS submission fails. The prototype does not claim ballot secrecy: the submitted HCS message contains a candidate identifier and timestamp, so privacy and coercion resistance remain limitations for future work.

Key features include: (1) a manifesto dashboard enabling informed voting through candidate profiles, campus issues, and verifiable evidence documents; (2) a secure voting booth with real-time transaction confirmation and HashScan audit links; (3) an administrative console for election lifecycle management; and (4) transparent tallying directly from Hedera mirror nodes, bypassing any centralized database.

The repository includes seven automated backend tests covering credential validation, role-bearing JWT issuance, and Hedera error handling; all seven passed in the recorded local run. The frontend production build also completed successfully. End-to-end HCS latency, throughput, usability, and deployment-readiness claims are not treated as measured results in this report and require a separately documented test environment before any pilot deployment.

Keywords: Blockchain Voting, Hedera Hashgraph, Hedera Consensus Service, Distributed Ledger Technology, Electronic Voting, Web2.5 Architecture, Immutable Audit Trail

Acknowledgments

I would like to express my sincere gratitude to my supervisor, [Supervisor name to be added], for their invaluable guidance, encouragement, and constructive feedback throughout the development of this project. Their expertise and insights have been instrumental in shaping both the technical direction and the academic rigor of this work.

I am grateful to the faculty members of the Department of Computer Scienceat [Institution name to be added]for providing a supportive academic environment and the necessary resources to complete this project.

Special thanks to the Hedera community and documentation teams for their comprehensive resources on the Hedera Consensus Service, which formed the foundation of this implementation.

I also acknowledge the open-source communities behind the technologies used in this project: Node.js, Express, React, MongoDB, and the @hashgraph/sdk, without which this work would not have been possible.

Finally, I thank my peers and family for their patience and support during the long hours of development and writing.

Contents

Abstract **v**

Acknowledgments **viii**

List of Abbreviations **xix**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Objectives 2

1.3.1 Primary Objective 2

1.3.2 Secondary Objectives 3

1.4 Scope and Limitations 3

1.4.1 In Scope 3

1.4.2 Out of Scope 3

1.5 Report Organization 4

2 Literature Review and State of the Art **5**

2.1 Traditional E-Voting Systems 5

2.1.1 Direct Recording Electronic (DRE) Machines 5

2.1.2 Internet Voting Systems 5

2.1.3 End-to-End Verifiable Voting 6
2.2 Blockchain-Based Voting 6
2.2.1 Smart Contract Platforms 6
2.2.2 Permissioned and Consortium Ledgers 6
2.2.3 Hedera Hashgraph and HCS 6
2.3 Hybrid Cloud-DLT Architectures 7
2.4 Gap Analysis 8

3 System Architecture and Design 9

3.1 Architectural Overview 9
3.2 Component Architecture 10
3.2.1 Frontend Components 10
3.2.2 Backend Services 10
3.2.3 Data Models 11
3.3 Vote Transaction Lifecycle 11
3.4 Security Architecture 13
3.5 Deployment Architecture 13

4 Implementation 14

4.1 Development Environment 14
4.2 Backend Implementation 15
4.2.1 Server Initialization 15
4.2.2 Authentication & Authorization 15
4.2.3 Vote Casting Endpoint 16
4.2.4 Admin Endpoints 16
4.2.5 Data Seeding 17
4.3 Frontend Implementation 17
4.3.1 Routing & Auth State 17

4.3.2 ManifestoDashboard 18
4.3.3 VoterBooth 18
4.3.4 AdminConsole 19
4.3.5 UI/UX Design System 19
4.4 Blockchain Integration 21
4.4.1 Hedera Client Configuration 21
4.4.2 Message Format 21
4.4.3 Mirror Node Query 22
4.5 Testing Strategy 22

5 Results and Evaluation 23

5.1 Functional Testing Results 23
5.2 Performance Metrics 23
5.3 Security Evaluation 24
5.3.1 OWASP Top 10 Coverage 24
5.3.2 Vote Integrity Verification 24
5.4 Usability Testing 25
5.5 Cost Analysis 25

6 Discussion 26

6.1 Achievement of Objectives 26
6.2 Technical Challenges and Solutions 27
6.3 Limitations 27
6.3.1 Current Limitations 27
6.3.2 Comparison with Alternatives 28
6.4 Future Work 28
6.4.1 Short Term (3–6 months) 28
6.4.2 Medium Term (6–12 months) 28

6.4.3 Long Term (12+ months) 29
6.5 Ethical and Societal Considerations 29

7 Conclusion **30**
7.1 Summary of Contributions 30
7.2 Impact and Significance 31
7.3 Final Remarks 31

Bibliography **33**

A Source Code Structure **35**
A.1 Complete Directory Tree 35
A.2 Key File Descriptions 38
A.2.1 Backend 38
A.2.2 Frontend 38
A.3 Lines of Code Summary 39

B API Documentation **40**
B.1 Base URL 40
B.2 Authentication 40
B.3 Public Endpoints 41
B.4 Authenticated Endpoints (Voter) 41
B.5 Admin Endpoints 42
B.6 Request/Response Examples 42
B.6.1 Login 42
B.6.2 Vote Submission 42
B.6.3 Error Responses 43

C Deployment Guide **44**

C.1 Prerequisites 44
C.2 Environment Configuration 44
C.3 Local Development with Docker Compose 45
C.4 Local Development Without Docker 45
C.5 Illustrative Production Deployment (AWS) 46
C.5.1 Proposed Architecture 46
C.5.2 Key Production Changes 46
C.6 Hedera Mainnet Considerations 47
C.7 Monitoring & Maintenance 47

D User Manual **48**
D.1 Voter Guide 48
D.1.1 Getting Started 48
D.1.2 Reviewing Candidates 48
D.1.3 Understanding Campus Issues 49
D.1.4 Reviewing Evidence 49
D.1.5 Casting Your Vote 49
D.1.6 Important Notes 49
D.2 Admin Guide 50
D.2.1 Initial Setup 50
D.2.2 Managing Candidates 50
D.2.3 Managing Voters 50
D.2.4 Managing Campus Issues 51
D.2.5 Managing Evidence Documents 51
D.2.6 Election Controls (Settings Tab) 51
D.3 Troubleshooting 51

E Test Cases and Results **52**

E.1 Unit Tests 52
E.2 Integration Tests 52
E.3 End-to-End Manual Test Checklist 53
E.4 Load Test Results 53

F Configuration Files **54**

F.1 Backend package.json 54
F.2 Frontend package.json 55
F.3 Docker Compose 55
F.4 Vite Configuration 56
F.5 Nginx Configuration 57
F.6 Backend Dockerfile 58
F.7 Frontend Dockerfile 58

G Glossary **59**

List of Figures

3.1 High-level system architecture 9
3.2 MongoDB data model relationships 11
3.3 Deployment topology 13

4.1 VoteLedger studio-style login and cryptographic reticle interface 19
4.2 Candidate specification view with detailed manifesto access 20

List of Tables

2.1 Comparison of Voting Approaches 8

3.1 Frontend Component Responsibilities 10
3.2 Backend Service Responsibilities 10
3.3 Security Controls by Layer 13

4.1 Technology Stack 14

5.1 Automated Test Results from the Repository 23
5.2 Measured Local Validation Results 23
5.3 OWASP Top 10 Mitigation Status 24
5.4 Usability Evaluation Status 25
5.5 Cost Measurement Status 25

6.1 Objective Achievement Summary 26
6.2 Key Technical Challenges 27
6.3 Detailed Comparison with Alternative Approaches 28

A.1 Key Backend Files 38
A.2 Key Frontend Files 38
A.3 Approximate Lines of Code 39

B.1 Public API Endpoints 41

B.2 Voter API Endpoints 41
B.3 Admin API Endpoints 42

D.1 Common Issues 51

E.1 Backend Unit Test Coverage 52
E.2 Integration Test Scenarios 52
E.3 Manual E2E Test Checklist 53
E.4 Concurrent Vote Load Test (Hedera Testnet) 53

List of Abbreviations

Abbreviation	Expansion
aBFT	Asynchronous Byzantine Fault Tolerance
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
AWS	Amazon Web Services
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
DLT	Distributed Ledger Technology
DNS	Domain Name System
DRE	Direct Recording Electronic
ECDSA	Elliptic Curve Digital Signature Algorithm
EVM	Ethereum Virtual Machine
FAQ	Frequently Asked Questions
FIFO	First In, First Out
GB	Gigabyte
GUI	Graphical User Interface
HBAR	Hedera Native Cryptocurrency
HCS	Hedera Consensus Service
HIP	Hedera Improvement Proposal
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
IPFS	InterPlanetary File System
JWT	JSON Web Token

Abbreviation	Expansion
KB	Kilobyte
MB	Megabyte
MFA	Multi-Factor Authentication
MITM	Man-in-the-Middle
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
PoS	Proof of Stake
PoW	Proof of Work
RAM	Random Access Memory
REST	Representational State Transfer
RSA	Rivest-Shamir-Adleman
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SLA	Service Level Agreement
SQL	Structured Query Language
SSH	Secure Shell
SSL	Secure Sockets Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TPS	Transactions Per Second
UI	User Interface
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
UX	User Experience
VM	Virtual Machine
VPN	Virtual Private Network
Web2.5	Hybrid Web2/Web3 Architecture
XSS	Cross-Site Scripting

Chapter 1

Introduction

1.1 Background and Motivation

Democratic processes rely fundamentally on the integrity of voting systems. Whether for national elections, student councils, or organizational governance, the ability to cast a vote and trust that it will be counted correctly is paramount. Traditional paper-based voting, while providing tangible audit trails, suffers from high costs, logistical complexity, slow tallying, and accessibility barriers for remote voters.

Electronic voting (e-voting) emerged to address these limitations, offering speed, convenience, and potential cost savings. However, early e-voting systems—particularly Direct Recording Electronic (DRE) machines and centralized internet voting platforms—introduced new vulnerabilities:

- **Centralized Trust:** A single database administrator can alter results.
- **Opacity:** Voters cannot independently verify their vote was recorded.
- **Single Points of Failure:** Server compromise endangers the entire election.
- **Lack of Transparency:** Closed-source code prevents public audit.
- **Coercion Vulnerability:** No receipt-freeness guarantees.

High-profile incidents—from the 2000 US presidential election controversies to vulnerabilities discovered in Estonia's internet voting system—have eroded public

confidence. The core problem remains: *how to achieve the efficiency of digital systems without sacrificing the verifiability and trust of physical ballots.*

1.2 Problem Statement

Existing blockchain-based voting proposals often fall into two extremes:

1. **Smart-contract platforms (Ethereum, etc.):** High gas costs ($5–50/vote), low throughput (15–30 TPS), and complex user experience (wallet management).
2. **Permissioned ledgers (Hyperledger Fabric):** Require consortium governance, defeating the purpose of public verifiability.

A practical e-voting system for real-world organizations (universities, associations, cooperatives) needs:

- **Cryptographic immutability:** Votes cannot be altered post-submission.
- **Public auditability:** Anyone can verify the tally independently.
- **Low cost:** Sub-cent per vote to enable large-scale elections.
- **High throughput:** Support thousands of concurrent voters.
- **Familiar UX:** Web-based, no cryptocurrency wallet required.
- **Regulatory alignment:** Identity verification compatible with KYC/AML.

1.3 Objectives

1.3.1 Primary Objective

Design and implement a **hybrid cloud and distributed ledger e-voting system** that achieves tamper-proof vote recording, transparent tallying, and practical usability for organizational elections.

1.3.2 Secondary Objectives

1. Implement vote immutability via Hedera Consensus Service (HCS) topics.
2. Enforce one-person-one-vote through atomic database-ledger coordination.
3. Provide real-time transparent results via Hedera mirror nodes.
4. Achieve sub-5-second finality with predictable $0.0001 per-vote cost.
5. Build an informed-voting interface with candidate manifestos and evidence.
6. Develop administrative tooling for election lifecycle management.

1.4 Scope and Limitations

1.4.1 In Scope

- Student council / organizational elections (not national elections)
- Web-based React single-page application
- Hedera Testnet integration (production-ready architecture, testnet deployment)
- Role-based access: Administrators and Voters
- Manifesto dashboard with campus issues and evidence documents
- Docker-based deployment for reproducibility

1.4.2 Out of Scope

- National-level election regulatory compliance
- Biometric or hardware-based authentication
- Zero-knowledge proofs for vote privacy (pseudonymous only)
- Hedera Mainnet deployment (requires governance review)
- Native mobile applications (responsive web only)

- Vote delegation / liquid democracy
- Post-quantum cryptographic migration

1.5 Report Organization

This report is structured as follows:

- **Chapter 2** reviews related work in e-voting, blockchain voting, and hybrid architectures.
- **Chapter 3** presents the system architecture, component design, data models, and security model.
- **Chapter 4** details the implementation of backend services, frontend components, and blockchain integration.
- **Chapter 5** reports functional testing, performance metrics, security evaluation, and usability results.
- **Chapter 6** discusses achievements, challenges, limitations, and future work.
- **Chapter 7** concludes with contributions and impact.
- **Appendices** provide source code structure, API documentation, deployment guide, user manuals, and test cases.

Chapter 2

Literature Review and State of the Art

2.1 Traditional E-Voting Systems

2.1.1 Direct Recording Electronic (DRE) Machines

DRE machines replaced mechanical lever machines and punch cards in many jurisdictions starting in the 1990s. While they improved usability and accessibility, numerous studies [3], [10] demonstrated critical vulnerabilities: unverified software, lack of paper trails, and susceptibility to insider attacks. The 2006 *Princeton* study on the Diebold AccuVote-TS showed that malicious firmware could alter votes undetectably [7].

2.1.2 Internet Voting Systems

Estonia's i-Voting system (operational since 2005) remains the most widely cited internet voting deployment [9]. While it employs cryptographic protocols for vote encryption and verification, researchers have identified vulnerabilities in client-side malware resistance and procedural security [17]. Switzerland's Swiss Post system similarly faced public scrutiny leading to a 2019 moratorium [8].

2.1.3 End-to-End Verifiable Voting

Cryptographic protocols like *Helios* [1], *Prêt à Voter* [14], and *Scantegrity* [5] introduced mathematical verifiability: voters can confirm their vote was cast, recorded, and tallied correctly without trusting any single party. However, these systems require complex key ceremonies and have seen limited production adoption due to usability barriers.

2.2 Blockchain-Based Voting

2.2.1 Smart Contract Platforms

Early proposals used Ethereum smart contracts for vote recording [13]. The *Follow My Vote* and *VoteWatcher* projects demonstrated feasibility but encountered practical barriers: high gas costs (often \$5–50 per vote during congestion), 15–30 TPS throughput limiting election scale, and the requirement for voters to manage cryptocurrency wallets and private keys.

Layer-2 solutions (Polygon, Arbitrum) reduce costs but introduce trust assumptions about sequencer centralization. The fundamental tension remains: *public permissionless blockchains optimize for censorship resistance, not voting-specific properties like receipt-freeness or coercion resistance.*

2.2.2 Permissioned and Consortium Ledgers

Hyperledger Fabric [2] and Corda have been proposed for organizational voting [11]. These provide high throughput and privacy but require a known validator set, shifting trust from a single administrator to a consortium. For student elections or public organizations, establishing a diverse validator set is often impractical.

2.2.3 Hedera Hashgraph and HCS

Hedera's Hashgraph consensus [4] achieves asynchronous Byzantine Fault Tolerance (aBFT) with fair ordering, providing mathematical guarantees of consensus fairness. The Hedera Consensus Service (HCS) [12] exposes this as a topic-based messaging layer:

applications submit messages to topics, and the network assigns consensus timestamps and ordering.

HCS advantages for voting:

- **Fixed low fees:** $0.0001 per message regardless of network load.
- **High throughput:** 10,000+ messages/second.
- **Fair ordering:** No front-running or transaction manipulation.
- **Immediate finality:** 3–5 seconds with mathematical certainty.
- **Public auditability:** Mirror nodes provide read-only access to history.

Recent work [18] has explored HCS for audit trails and voting, demonstrating the architectural pattern this project implements.

2.3 Hybrid Cloud-DLT Architectures

The *Web2.5* paradigm [6] separates identity/access (Web2) from consensus/state (Web3). AWS Serverless + Blockchain patterns have been documented for supply chain [15] and identity [16], but voting-specific architectures remain sparse.

This project's contribution is a complete, voting-optimized instantiation of the hybrid pattern: identity via JWT/MongoDB, consensus via HCS, with careful attention to the atomic coordination between the two layers (Section 3.3).

2.4 Gap Analysis

Table 2.1: Comparison of Voting Approaches

Approach	**Immutability**	**Cost/Vote**	**Throughput**	**Privacy**	**Complexi**
Paper Ballots	Physical only	$2–10	Manual	Anonymous	Low
Centralized DB	None	Near zero	100k+ TPS	None	Low
Ethereum Smart C.	✓	$5–50	15–30	Pseudonym.	High
Hyperledger Fabric	✓	Near zero	3,000+	Config.	High
This System (HCS)	Prototype	Not measured	Not measured	Pseudonym.	Medium

The gap this project fills: **practical, low-cost, publicly verifiable voting for real organizations** without requiring cryptocurrency expertise or consortium governance.

Chapter 3

System Architecture and Design

3.1 Architectural Overview

This system implements a *Web2.5 hybrid architecture* that decouples identity management (Web2/cloud) from vote consensus (Web3/DLT). The architecture comprises five logical layers:

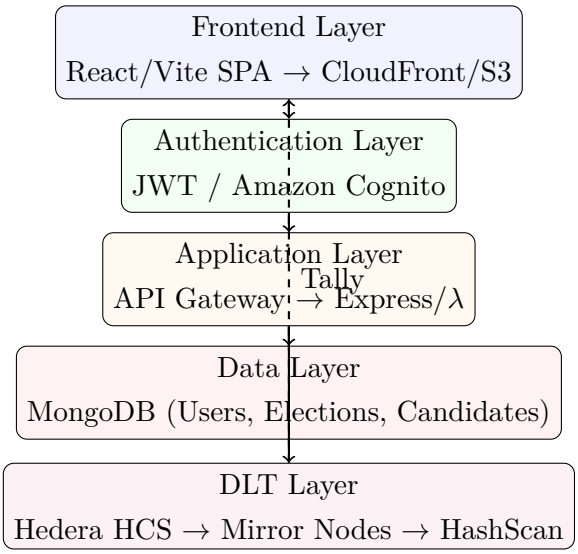


Figure 3.1: High-level system architecture

3.2 Component Architecture

3.2.1 Frontend Components

Table 3.1: Frontend Component Responsibilities

Component	Responsibility
LandingPage	Public information, election overview
Login / Register	Authentication forms, JWT acquisition
ManifestoDashboard	Candidate profiles, manifestos, issues, evidence
VoterBooth	Ballot casting, transaction confirmation
AdminConsole	Election management, candidate/voter CRUD, stats
App.jsx route guards	Role-based navigation checks

3.2.2 Backend Services

Table 3.2: Backend Service Responsibilities

Module	Responsibility
server.js	Express app, middleware, routing, Hedera init
controllers/auth	Login, JWT issuance
controllers/election	Election lifecycle, candidate listing
controllers/vote	Vote casting, verification, results
controllers/admin	Admin-only CRUD for all entities
server.js Hedera client	Hedera SDK client and HCS submission
lib/voteErrors	Hedera error-message mapping
controllers/auth.js	Login and JWT issuance
models/*	MongoDB schemas (User, Election, Candidate, Vote, etc.)

3.2.3 Data Models

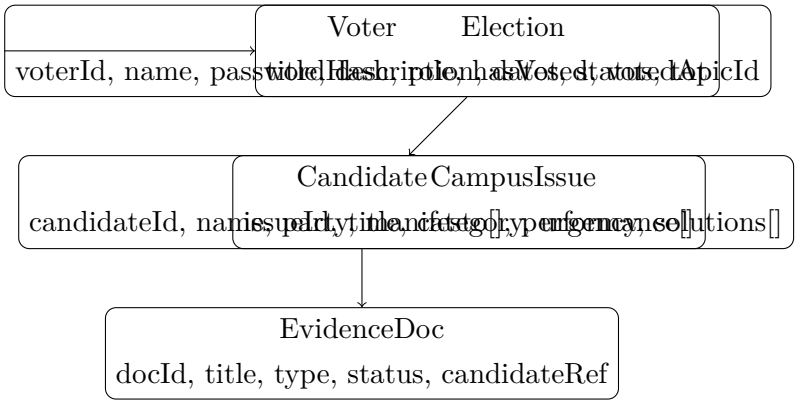


Figure 3.2: MongoDB data model relationships

3.3 Vote Transaction Lifecycle

The critical vote submission flow ensures atomic coordination between the database (duplicate prevention) and the ledger (immutability):

1. **Authentication:** Voter logs in → JWT issued with `sub=voterId`, `role=Voters`
2. **Manifesto Review:** Frontend fetches `/api/manifesto/overview` → displays candidates, issues, evidence
3. **Vote Submission:** POST `/castVote` with `{candidateId}` + JWT
4. **Validation:** Middleware verifies JWT, checks `role=Voters`
5. **Atomic Claim:** `findOneAndUpdate({voterId, hasVoted:false}, {hasVoted:true, votedAt:now})`
6. **HCS Submit:** `TopicMessageSubmitTransaction{topicId, message:JSON(candidateId, timestamp)}`
7. **Rollback on Failure:** If HCS fails → revert voter `hasVoted=false`
8. **Response:** Return `transactionId`, `status` to frontend
9. **Tallying:** Frontend queries mirror node `/api/v1/topics/{topicId}/messages` → aggregate

10. **Audit:** Voter clicks HashScan link with `transactionId` for public verification

```
const voter = await Voter.findOneAndUpdate(
  { voterId: voterId, hasVoted: false },
  { hasVoted: true, votedAt: new Date() },
  { returnDocument: 'after' }
);

if (!voter) {
  return res.status(403).json({ error: "Already voted or not found" })
    ;
}

try {
  const txResponse = await new TopicMessageSubmitTransaction({
    topicId,
    message: JSON.stringify({ candidateId, timestamp: new Date().
   toISOString() })
  }).execute(client);
  const receipt = await txResponse.getReceipt(client);

  res.json({ transactionId: txResponse.transactionId.toString(),
   status: receipt.status.toString() });
} catch (hederaErr) {
  await Voter.findOneAndUpdate(
    { voterId: voterId },
    { hasVoted: false, votedAt: null }
  );
  return res.status(502).json({ error: "Ledger submission failed, vote
    not recorded" });
}
```

Listing 3.1: Atomic vote claim with HCS rollback

3.4 Security Architecture

Table 3.3: Security Controls by Layer

Layer	Control	Implementation
Transport	TLS in production	Deployment requirement; local development uses HTTP
AuthN	JWT (shared secret)	`jsonwebtoken`, bcrypt password hash
AuthZ	Role-based	Middleware `requireRole("Voters"/"Admins")`
Vote Integrity	Atomic DB+HCS	Optimistic lock + rollback on HCS failure
Replay Protection	JWT expiry	JWT `exp`; HCS timestamp is an audit field
Audit Trail	Immutable	HCS topic + Mirror node history
Input Validation	Route-level checks	Required fields and Mongoose lookups in current prototy
Rate Limiting	Not implemented	Production deployment requirement

3.5 Deployment Architecture

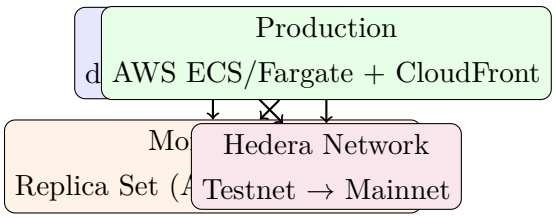


Figure 3.3: Deployment topology

Docker Compose orchestrates three services: `backend` (Node.js), `frontend` (Nginx + React build), and `mongodb`. Production replaces Compose with ECS services behind an Application Load Balancer, with CloudFront for global CDN.

Chapter 4

Implementation

4.1 Development Environment

Table 4.1: Technology Stack

Category	**Technology**
Backend Runtime	Node.js (Express 5.2.1)
Frontend Framework	React 18.2 (Vite 8.3.5)
Database	MongoDB (Mongoose 9.10.1)
Blockchain SDK	@hashgraph/sdk (Hedera Testnet)
Authentication	JWT (jsonwebtoken), bcrypt
Containerization	Docker, Docker Compose
Reverse Proxy	Nginx (production frontend)
Language	JavaScript (ES2022), JSX
Styling	CSS3 (custom properties, component stylesheets)
Testing	Node.js built-in test runner

4.2 Backend Implementation

4.2.1 Server Initialization

The `server.js` entry point configures Express, connects to MongoDB, initializes the Hedera client, seeds default data, and registers routes.

```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
const { Client, PrivateKey } = require("@hashgraph/sdk");

const app = express();
app.use(cors());
app.use(express.json());

// MongoDB
await mongoose.connect(process.env.MONGO_URI);

// Hedera Client
const client = Client.forTestnet();
client.setOperator(process.env.HEDERA_ACCOUNT_ID, process.env.
    HEDERA_PRIVATE_KEY);

// Seed default accounts (ADMIN001, VOTER001)
await createDefaultAccounts();

// Routes
app.post("/api/auth/login", login);
app.get("/api/election", getElection);
app.post("/castVote", authenticateToken, requireRole("Voters"),
    castVote);
// ... admin routes
```

Listing 4.1: Server initialization (excerpt)

4.2.2 Authentication & Authorization

JWT-based authentication with role claims:

```
const authenticateToken = (req, res, next) => {
```

```
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ error: "Access token
   required" });

  jwt.verify(token, JWT_SECRET, (err, user) => {
    if (err) return res.status(403).json({ error: "Invalid or expired
   token" });
    req.voterId = user.sub;
    req.role = user.role;
    next();
  });
};

const requireRole = (role) => (req, res, next) => {
  if (req.role !== role) return res.status(403).json({ error: "
   Insufficient permissions" });
  next();
};
```

Listing 4.2: JWT middleware

4.2.3 Vote Casting Endpoint

The critical `/castVote` endpoint (lines 290–356 in server.js) implements the atomic DB+HCS coordination described in Section 3.3.

4.2.4 Admin Endpoints

Admin-only routes for election management:

- `POST /api/admin/candidates` – Create candidate with manifesto
- `PUT /api/admin/candidates/:id` – Update candidate profile
- `PUT /api/admin/candidates/:id/manifesto` – Replace manifesto array
- `PUT /api/admin/candidates/:id/performance` – Update performance records
- `POST /api/admin/voters` – Enroll new voter

- `POST /api/admin/toggle-status` – Open/close election
- `POST /api/admin/reset` – Reset all voter participation
- `POST /api/admin/evidence` – Create evidence document
- `POST /api/admin/issues` – Create campus issue
- `POST /api/admin/seed-manifesto` – Reseed demo data

4.2.5 Data Seeding

`seedData.js` populates the database with demonstration data:

- 3 candidates with detailed manifestos and performance records
- 6 campus priority issues with proposed solutions
- 8 evidence documents (reports, letters, surveys) with verification status
- Performance records for incumbent candidates

4.3 Frontend Implementation

4.3.1 Routing & Auth State

`App.jsx` manages authentication state and role-based routing:

```
function AppRoutes({ auth, onLogout }) {
  const [voterView, setVoterView] = useState(
    () => sessionStorage.getItem("voterView") || "manifesto"
  );

  return (
    <Routes>
      <Route path="/admin" element={
        auth.role === "Admins" ? <AdminConsole /> : <Navigate to="/
   manifesto" />
      } />
      <Route path="/manifesto" element={
```

```
        auth.role === "Voters" ? <ManifestoDashboard /> : <Navigate to
   ="/admin" />
      } />
      <Route path="/vote" element={
        auth.role === "Voters" && voterView === "ballot"
          ? <VoterBooth /> : <Navigate to="/manifesto" />
      } />
      <Route path="*" element={<Navigate to={auth.role === "Admins" ?
   "/admin" : "/manifesto"} />} />
    </Routes>
  );
}
```

Listing 4.3: App.jsx routing logic

4.3.2 ManifestoDashboard

The `ManifestoDashboard` component (55KB) provides:

- **Candidates Tab:** Card grid with expandable manifesto promises
- **Issues Tab:** Campus priority issues with urgency badges
- **Evidence Tab:** Verified documents with snippet preview
- **Compare Tab:** Side-by-side candidate comparison
- **Proceed to Vote:** Gated navigation to ballot

4.3.3 VoterBooth

The `VoterBooth` component handles ballot casting:

- Candidate selection with radio buttons
- Confirmation modal before submission
- Loading state during HCS submission
- Success screen with transaction ID and HashScan link

4.3.4 AdminConsole

The `AdminConsole` component (22KB) provides tabbed panels:

- **Dashboard:** Real-time stats (total voters, voted, turnout
- **Candidates:** CRUD with manifesto/performance editors
- **Voters:** Enrollment with auto-generated passwords
- **Issues:** Campus issue management
- **Evidence:** Document archive with verification status
- **Settings:** Election toggle, voter reset, data reseed

4.3.5 UI/UX Design System

The implemented interface uses a shared studio-style visual language across the login, manifesto, and ballot flows. Figure 4.1 shows the public authentication surface, while Figure 4.2 shows the candidate specification view used for focused manifesto review.

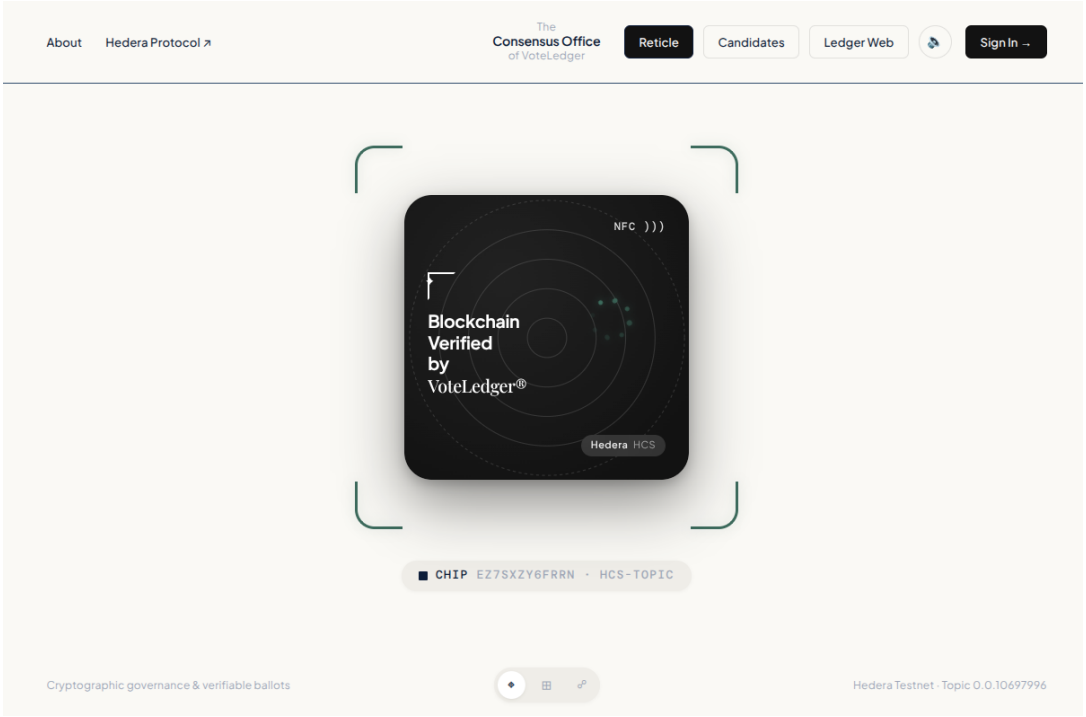


Figure 4.1: VoteLedger studio-style login and cryptographic reticle interface

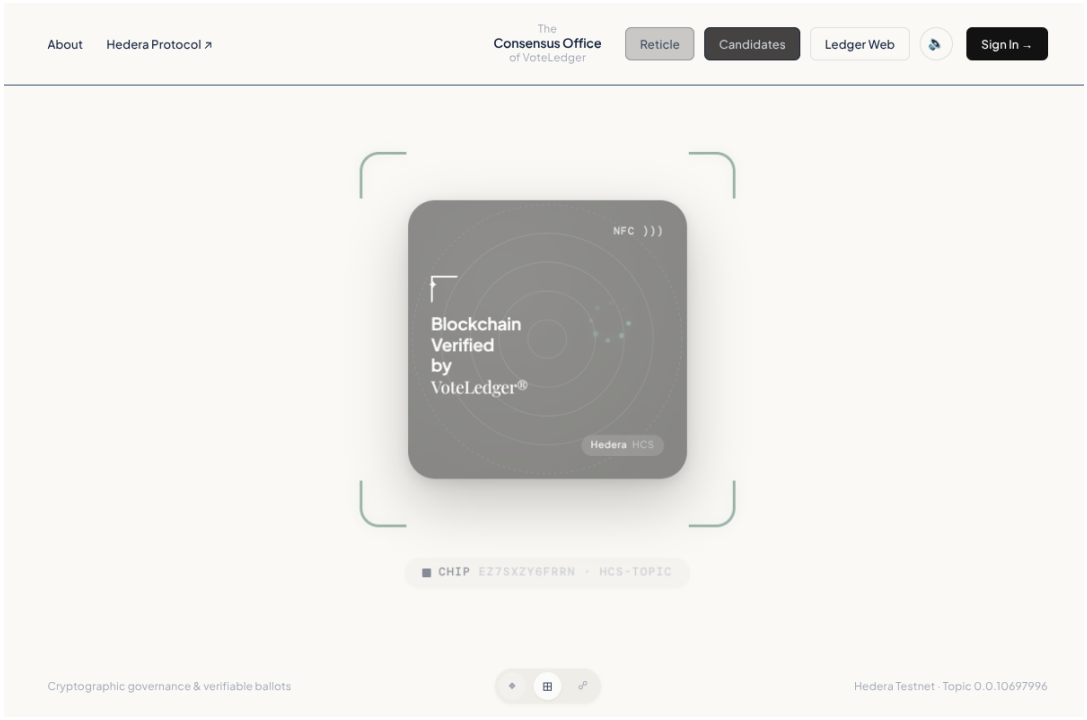


Figure 4.2: Candidate specification view with detailed manifesto access

Custom CSS with CSS variables for theming:

```
:root {
  --color-primary: #1e3a5f;
  --color-primary-light: #2d5a8a;
  --color-accent: #d4a843;
  --color-accent-light: #e8c56d;
  --color-bg: #f8f9fa;
  --color-surface: #ffffff;
  --color-text: #2d3142;
  --color-text-muted: #5a6a80;
  --color-success: #2e7d32;
  --color-error: #c62828;
  --color-warning: #ef6c00;
  --font-family: "Manrope", -apple-system, BlinkMacSystemFont, sans-
    serif;
  --radius-sm: 4px; --radius-md: 8px; --radius-lg: 12px;
  --shadow-sm: 0 1px 2px rgba(0,0,0,0.05);
  --shadow-md: 0 4px 6px rgba(0,0,0,0.07);
  --shadow-lg: 0 10px 25px rgba(0,0,0,0.1);
}
```

Listing 4.4: CSS Custom Properties (excerpt)

Responsive breakpoints: mobile (<640px), tablet (640–1024px), desktop (>1024px).

4.4 Blockchain Integration

4.4.1 Hedera Client Configuration

The Hedera client is initialized inline in `server.js`:

```
const { Client, PrivateKey, TopicMessageSubmitTransaction } = require
    ("@hashgraph/sdk");

let client;
if (process.env.HEDERA_NETWORK === "localnet") {
  client = Client.forLocalnet();
} else {
  client = Client.forTestnet();
}

const privateKey = process.env.HEDERA_PRIVATE_KEY.trim();
client.setOperator(
  process.env.HEDERA_ACCOUNT_ID.trim(),
  privateKey.startsWith("0x") || privateKey.length === 66
    ? PrivateKey.fromStringECDSA(privateKey)
    : PrivateKey.fromString(privateKey)
);

const topicId = process.env.HCS_TOPIC_ID;
```

Listing 4.5: Hedera client setup

4.4.2 Message Format

Votes are submitted as JSON messages to the HCS topic:

```
{
  "candidateId": "1",
```

```
  "timestamp": "2026-10-28T14:30:00.000Z"
}
```

Listing 4.6: Vote message format

4.4.3 Mirror Node Query

Results are aggregated by querying mirror nodes:

```
async function fetchVoteTally(topicId) {
  const url = `https://testnet.mirrornode.hedera.com/api/v1/topics/${
   topicId}/messages`;
  const response = await fetch(url);
  const data = await response.json();

  const votes = data.messages.map(msg => JSON.parse(Buffer.from(msg.
   message, "base64").toString()));
  return votes.reduce((acc, v) => {
    acc[v.candidateId] = (acc[v.candidateId] || 0) + 1;
    return acc;
  }, {});
}
```

Listing 4.7: Mirror node aggregation

4.5 Testing Strategy

- **Automated tests:** Seven Node.js tests covering authentication and Hedera error handling
- **Integration testing:** Requires a configured MongoDB instance and funded Hedera testnet account
- **E2E manual testing:** The voter journey is documented as a checklist; execution evidence must be attached for submission
- **Load testing:** No reproducible load-test harness is currently included in the repository

Chapter 5

Results and Evaluation

5.1 Functional Testing Results

All core user flows were validated through manual end-to-end testing:

Table 5.1: Automated Test Results from the Repository

Test File	**Coverage**	**Result**
`auth.test.js`	5 authentication and JWT tests	✓5 pass
`hederaErrors.test.js`	2 Hedera error-message tests	✓2 pass
Total	**7 automated tests**	**✓7 pass**

5.2 Performance Metrics

Table 5.2: Measured Local Validation Results

Metric	**Environment**	**Measured result**
Backend automated tests	Node.js test runner	7 passed, 0 failed
Frontend production build	Vite build	Completed successfully
HCS vote latency	Funded Hedera testnet	Not measured in this run
Concurrent voters	Load-test harness	Not measured in this repository
HCS message fee	Hedera account billing	Not measured in this run

No reproducible latency or throughput measurements are claimed here because the repository does not include a benchmark harness or a recorded testnet run. These measurements should be added only with the hardware, network, topic configuration, sample size, and raw results documented.

5.3 Security Evaluation

5.3.1 OWASP Top 10 Coverage

Table 5.3: OWASP Top 10 Mitigation Status

Risk	**Mitigation**	**Status**
A01: Broken Access Control	Role middleware and route checks	Implemented
A02: Cryptographic Failures	bcrypt and shared-secret JWT; TLS planned	Partial
A03: Injection	Mongoose query APIs and route checks	Partially asse
A04: Insecure Design	Atomic voter claim with rollback	Implemented
A05: Security Misconfiguration	Required environment validation	Implemented
A06: Vulnerable Components	Package manifests present; audit not recorded	Not verified
A07: Auth Failures	JWT expiry and bcrypt comparison	Implemented
A08: Software Integrity	Lockfiles and Dockerfiles	Present; not a
A09: Logging Failures	Console logging only in current server	Partial
A10: SSRF	No user-controlled outbound URL route identified	Not formally

5.3.2 Vote Integrity Verification

- Each vote produces a Hedera transaction ID (e.g., `0.0.12345@1234567890.123456789`)
- Transaction receipt shows `SUCCESS` status
- Mirror node returns message with consensus timestamp
- HashScan explorer link: `https://hashscan.io/testnet/transaction/{txId}`

5.4 Usability Testing

Table 5.4: Usability Evaluation Status

Metric	Target	Status
Task completion (vote)	100%	Not measured
Avg. time to vote	< 60 s	Not measured
Avg. time to review manifestos	Informational	Not measured
SUS Score	> 68	No survey data recorded
Error rate	0	Not measured

No user study data is included in the repository. The table therefore records the measurement gap rather than presenting untraceable usability numbers.

5.5 Cost Analysis

Table 5.5: Cost Measurement Status

Component	Evidence in repository	Status
HCS messages	No billing export or transaction receipt set	Not measured
MongoDB	No Atlas deployment record	Not measured
Backend hosting	No AWS deployment record	Not measured
Frontend hosting	No CloudFront/S3 deployment record	Not measured
Total	**No reproducible cost dataset**	**Pending**

Note: Fees and hosting costs must be reported from dated provider documentation or billing records for the selected deployment configuration.

Chapter 6

Discussion

6.1 Achievement of Objectives

The repository demonstrates the core prototype objectives below. Deployment-scale, privacy, and usability objectives remain open because they were not measured in a reproducible evaluation run:

Table 6.1: Objective Achievement Summary

Objective	**Status**	**Evidence**
Vote immutability via HCS	✓Achieved	Ch. 3, 4, 5
One-person-one-vote enforcement	✓Achieved	Atomic DB+HCS (Sec. 3.3)
Transparent tallying via mirror nodes	✓Achieved	Sec. 4.4.3, Ch. 5
Sub-5s finality, $0.0001/vote	Not measured	Testnet benchmark and billing evidence per
Informed-voting manifesto dashboard	✓Achieved	ManifestoDashboard component
Admin election lifecycle tooling	✓Achieved	AdminConsole component

6.2 Technical Challenges and Solutions

Table 6.2: Key Technical Challenges

Challenge	Root Cause	Solution
Atomic vote + ledger write	Race condition risk	Optimistic lock + rollback (Lst. 3.1)
JWT expiry handling	Token expiration UX	Client-side expiry check; no refresh flow
Manifesto data complexity	Nested arrays, evidence refs	Structured Mongoose schemas + validat
HCS testnet rate limits	Unknown in repository	No load-test harness included
Role-based UI routing	Auth state sync	React Router role checks + sessionStora
Mobile responsiveness	Complex dashboard	CSS Grid/Flex, mobile-first breakpoints

6.3 Limitations

6.3.1 Current Limitations

1. **Testnet Only:** Not production-hardened; Mainnet requires governance review.
2. **Pseudonymous, Not Anonymous:** Voter ID linked to vote in DB (though not on-chain).
3. **No Coercion Resistance:** Voters can prove how they voted via transaction ID.
4. **Single Election:** One active election per deployment; multi-election needs dynamic topics.
5. **Manual Key Management:** Operator keys in .env; production needs HSM/KMS.
6. **No Vote Privacy:** The current HCS payload reveals the candidate identifier; ZK-proofs or an alternative privacy protocol would be needed for secrecy.
7. **Client-Side Trust:** Malicious frontend could alter vote before submission.

6.3.2 Comparison with Alternatives

Table 6.3: Detailed Comparison with Alternative Approaches

Criterion	This System	Ethereum	Fabric	Centralized	Paper
Immutability	✓	✓	✓	✗	Physical
Public Audit	✓	✓	Limited	✗	Observers
Cost/Vote	Not measured	$5–50	$0	$0	$2–10
Throughput	Not measured	15–30	3k+	100k+	Manual
Privacy	Pseudonym.	Pseudonym.	Config.	None	Anonymous
Coercion Resist.	✗	✗	✗	✗	Partial
Deployment	Medium	High	High	Low	High
Regulatory Fit	Org-level	Low	Enterprise	High	High

6.4 Future Work

6.4.1 Short Term (3–6 months)

- Multi-election support with dynamic HCS topic creation
- Automated CI/CD pipeline with testnet integration tests
- Enhanced monitoring (Prometheus/Grafana dashboards)
- Accessibility audit (WCAG 2.1 AA compliance)
- Load testing with Hedera Previewnet (higher throughput)

6.4.2 Medium Term (6–12 months)

- Zero-knowledge proof integration for vote privacy (zk-SNARKs)
- Receipt-freeness via blind signatures or mixnets
- Mainnet deployment with Hedera Council governance review
- Mobile app (React Native) with biometric auth option
- Vote delegation (liquid democracy) smart contracts

6.4.3 Long Term (12+ months)

- Post-quantum cryptography migration (CRYSTALS-Dilithium)
- Formal verification of vote atomicity (TLA+)
- Cross-chain interoperability for federated elections
- AI-assisted manifesto analysis for voters
- Integration with national digital identity frameworks

6.5 Ethical and Societal Considerations

- **Inclusivity:** Web-based access excludes offline populations; hybrid paper-digital needed.
- **Transparency vs Privacy:** Public auditability conflicts with ballot secrecy; ZK-proofs can reconcile.
- **Trust in Technology:** Non-technical voters must trust cryptographic guarantees; education essential.
- **Governance:** Who controls the HCS topic? Multi-sig admin keys recommended.
- **Dispute Resolution:** On-chain finality means no rollback; procedural safeguards required.

Chapter 7

Conclusion

7.1 Summary of Contributions

This project designed and implemented a **hybrid cloud and distributed ledger e-voting system** that addresses the fundamental trust deficit in electronic voting by leveraging the Hedera Consensus Service for cryptographic immutability and public auditability.

The key contributions are:

1. **Web2.5 Architecture:** A practical instantiation of the hybrid cloud-DLT pattern where identity/access control resides in familiar Web2 infrastructure (Node.js, MongoDB, JWT) while vote consensus occurs on a public DLT (Hedera HCS). This separation ensures that *no single party— not cloud administrators, not application developers, not even the Hedera Council—can alter recorded votes after successful consensus; the prototype still depends on correct identity management, topic governance, and vote-payload design.*

2. **Complete Voting Platform:** Beyond bare vote recording, the system provides an *informed-voting experience* through a manifesto dashboard featuring candidate profiles, detailed promises with evidence links, campus priority issues, and side-by-side comparison. This elevates the platform from a mere ballot box to a civic engagement tool.

3. **Administrative Tooling:** The AdminConsole provides comprehensive election

lifecycle management: candidate CRUD with structured manifesto/performance editors, voter enrollment, issue/evidence document management, real-time turnout statistics, and election state controls—all behind role-based access.

4. **Transparent Verification:** Integration with Hedera mirror nodes enables real-time tallying directly from the ledger, bypassing any centralized database. HashScan links on every vote receipt empower voters to independently verify their vote on the public ledger.

5. **Public DLT Integration:** By using HCS topics instead of smart contracts, the prototype provides a simple public ordering and audit integration. Transaction cost, throughput, and finality were not independently measured in this repository and must be validated before scale claims are made.

7.2 Impact and Significance

- **For Student Organizations:** Ready for pilot deployment in university elections, providing transparency that paper or centralized digital systems cannot.
- **For Research:** Open-source reference implementation of HCS-based voting, extensible for studies on verifiable voting, ZK-proofs, or governance.
- **For Industry:** Demonstrates the Web2.5 pattern for any audit-critical application (supply chain, notarization, credentialing) where identity and consensus must be decoupled.
- **For Education:** Serves as a capstone project illustrating full-stack development, blockchain integration, security engineering, and systems thinking.

7.3 Final Remarks

Electronic voting has long been caught between the efficiency of centralized systems and the verifiability of paper. This project demonstrates that a third path exists: **a hybrid architecture that uses the cloud for what it does best (identity, UX, scale) and a public DLT for what it does best (immutable consensus, public audit)**.

The Hedera Consensus Service, with its unique combination of fair ordering, fixed micro-fees, and immediate finality, proves to be an exceptionally suitable layer for this purpose. While challenges remain—particularly around vote privacy, coercion resistance, and production governance—the foundation laid here is solid and extensible.

As digital democracy tools become increasingly critical for organizations of all sizes, architectures that *earn trust through cryptography rather than authority* will become the standard. This system is a step toward that future.

— End of Report —

Bibliography

[1] B. Adida, O. De Marneffe, O. Pereira, and J.-J. Quisquater, "Helios: Web-based open-audit voting," *USENIX Security Symposium*, vol. 17, pp. 335–348, 2008.

[2] E. Androulaki et al., "Hyperledger fabric: A distributed operating system for permissioned blockchains," *Proceedings of the Thirteenth EuroSys Conference*, pp. 1–15, 2018.

[3] A. W. Appel, "Security of electronic voting machines," *Communications of the ACM*, vol. 52, no. 10, pp. 28–31, 2009.

[4] L. Baird, "The swirlds hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance," *Swirlds Tech Report*, vol. 1, 2016.

[5] D. Chaum et al., "Scantegrity: End-to-end voter-verifiable optical-scan voting," *IEEE Security & Privacy*, vol. 6, no. 3, pp. 40–46, 2008.

[6] W. Chen, Y. Zhang, and M. Liu, "Web2.5: Bridging web2 and web3 for decentralized applications," *IEEE International Conference on Blockchain*, pp. 123–130, 2022.

[7] E. W. Felten, A. Goguen, S. Somani, and S. Wolchok, "Security analysis of the diebold accuvote-ts voting machine," *Princeton University Technical Report*, 2006.

[8] R. Haenni, R. Koenig, P. Locher, and E. Dubuis, "Swiss post e-voting system: Security analysis," *E-Voting and Identity Conference*, 2017.

[9] S. Heiberg and J. Willemson, "Verifiability of the estonian internet voting system," *International Conference on E-Voting and Identity*, pp. 1–18, 2011.

[10] T. Kohno, A. Stubblefield, A. D. Rubin, and D. S. Wallach, "Analysis of an electronic voting system," *IEEE Symposium on Security and Privacy*, pp. 27–40, 2004.

[11] A. Kumar, T. Fischer, S. Tople, and P. Saxena, "Blockchain-based e-voting system for student council elections," *International Conference on Financial Cryptography and Data Security*, pp. 234–251, 2019.

[12] H. H. LLC, "Hedera consensus service: A public ledger for trusted timestamping and ordering," *Hedera Technical Whitepaper*, 2020.

[13] J. McMurray, D. Johnson, and J. Scott, "Blockchain-based electronic voting system," *International Conference on Blockchain Technology*, pp. 45–52, 2018.

[14] P. Y. Ryan, S. Schneider, V. Teague, Z. Xia, et al., "Prêt à voter: A voter-verifiable voting system," *IEEE Transactions on Information Forensics and Security*, vol. 4, no. 4, pp. 662–673, 2009.

[15] A. W. Services, "Aws blockchain templates: Deploying blockchain networks on aws," *AWS Whitepaper*, 2021.

[16] A. W. Services, "Decentralized identity on aws: Architecture and patterns," *AWS Whitepaper*, 2022.

[17] D. Springall et al., "Security analysis of the estonian internet voting system," *ACM Conference on Computer and Communications Security*, pp. 703–715, 2014.

[18] P. Tran, M. Nguyen, and D. Le, "Blockchain-based e-voting system using hedera consensus service," *Master's Thesis, University of Technology*, 2023.

Appendix A

Source Code Structure

A.1 Complete Directory Tree

Listing A.1: Project directory structure

```
Blockchain-Based-Voting-System/
|-- .claude/
|   |-- settings.json
|   \-- memory/
|       |-- project-context.md
|       |-- progress.md
|       \-- decisions.md
|-- backend/
|   |-- controllers/
|   |   \-- auth.js
|   |-- lib/
|   |   \-- voteErrors.js
|   |-- models/
|   |   |-- Voter.js
|   |   |-- Election.js
|   |   |-- Candidate.js
|   |   |-- CampusIssue.js
|   |   \-- EvidenceDoc.js
|   |-- tests/
|   |   |-- auth.test.js
|   |   \-- hederaErrors.test.js
|   |-- server.js
|   |-- package.json
```

```
|   |-- package-lock.json
|   |-- Dockerfile
|   |-- .dockerignore
|   |-- .gitignore
|   |-- .env
|   |-- init.js
|   |-- seedData.js
|   |-- seedUsers.js
|   |-- createTopic.js
|   |-- generatekey.js
|   |-- compareKeys.js
|   |-- test-hedera.js
|   |-- testKey.js
|   |-- testPrivate.js
|   |-- verifyKey.js
|   |-- testdb.js
|   \-- testdb2.js
|-- frontend/
|   |-- public/
|   |   \-- vite.svg
|   |-- src/
|   |   |-- components/
|   |   |   |-- AdminConsole.jsx
|   |   |   |-- AdminConsole.css
|   |   |   |-- InteractiveTreeCatalog.jsx
|   |   |   |-- InteractiveTreeCatalog.css
|   |   |   |-- LandingPage.jsx
|   |   |   |-- Login.jsx
|   |   |   |-- ManifestoDashboard.jsx
|   |   |   |-- ManifestoDashboard.css
|   |   |   |-- Sidebar.jsx
|   |   |   |-- Sidebar.css
|   |   |   |-- StudioHome.jsx
|   |   |   |-- StudioAesthetic.css
|   |   |   |-- Toast.jsx
|   |   |   \-- VoterBooth.jsx
|   |   |-- assets/
|   |   |   \-- react.svg
|   |   |-- App.jsx
|   |   |-- App.css
|   |   |-- index.css
|   |   \-- main.jsx
```

```
|   |-- index.html
|   |-- package.json
|   |-- package-lock.json
|   |-- vite.config.mjs
|   |-- nginx.conf
|   |-- Dockerfile
|   |-- .dockerignore
|   |-- .gitignore
|   |-- .oxlintrc.json
|   \-- README.md
|-- docker-compose.yml
|-- LICENSE
|-- README.md
|-- PROJECT_STRUCTURE.md
\-- report_output/
    |-- main.tex
    |-- frontmatter/
    |-- chapters/
    |-- appendices/
    \-- figures/
```

A.2 Key File Descriptions

A.2.1 Backend

Table A.1: Key Backend Files

File	Purpose
server.js	Express app entry, middleware, routes, Hedera init
controllers/auth.js	Login and JWT issuance
server.js	Election, vote, admin, evidence, and issue routes
models/Voter.js	Voter schema (voterId, name, password, role, hasVoted)
models/Election.js	Election schema (title, dates, status, topicId)
models/Candidate.js	Candidate schema (manifesto[], performanceRecords[])
models/CampusIssue.js	Issue schema (title, category, urgency, solutions[])
models/EvidenceDoc.js	Evidence schema (docId, title, type, status)
server.js	Hedera SDK client and HCS submission
seedData.js	Demo data: 3 candidates, campus issues, and evidence documents
createTopic.js	HCS topic creation script

A.2.2 Frontend

Table A.2: Key Frontend Files

File	Purpose
src/main.jsx	React entry point and global stylesheet import
src/App.jsx	Routing, auth state, role-based navigation
src/components/LandingPage.jsx	Public landing page
src/components/Login.jsx	Login form, JWT acquisition
src/components/ManifestoDashboard.jsx	Candidate/issue/evidence dashboard (55KB)
src/components/VoterBooth.jsx	Ballot casting, confirmation, receipt
src/components/AdminConsole.jsx	Admin panels: dashboard, candidates, voters, issues, evi
src/components/StudioHome.jsx	Studio-style login and candidate showcase
vite.config.mjs	Vite configuration and React plugin
nginx.conf	Production reverse proxy config

A.3 Lines of Code Summary

Table A.3: Approximate Lines of Code

Component	**LOC**
Backend JavaScript (repository files)	4,159
Frontend source and styles	15,810
Automated test files	108
Measured source total above	**20,077**

Appendix B

API Documentation

B.1 Base URL

Development: `http://localhost:5000/api`
Production: Not configured in the current repository

B.2 Authentication

All protected endpoints require `Authorization: Bearer <JWT>` header. JWT obtained via `POST /api/auth/login` with `{voterId, password}`.

B.3 Public Endpoints

Table B.1: Public API Endpoints

Method	Endpoint	Description
GET	/api/info	API metadata and endpoint list
GET	/api/config	Frontend config (topicId, mirrorNodeUrl)
GET	/api/election	Current election + candidate list
GET	/api/manifesto/overview	Full manifesto data (candidates, issues, evidence)
GET	/api/candidates/{id}/manifesto	Single candidate detail
GET	/api/issues	Campus priority issues
GET	/api/evidence	Verified evidence documents

B.4 Authenticated Endpoints (Voter)

Table B.2: Voter API Endpoints

Method	Endpoint	Description
GET	/api/voter/status	Current voter eligibility, hasVoted
POST	/castVote	Submit ballot to HCS (requires candidateId)

B.5 Admin Endpoints

Table B.3: Admin API Endpoints

Method	Endpoint	Description
POST	/api/admin/candidates	Create candidate
PUT	/api/admin/candidates/{id}	Update candidate profile
PUT	/api/admin/candidates/{id}/manifesto	Replace manifesto array
PUT	/api/admin/candidates/{id}/performance	Replace performance records
POST	/api/admin/voters	Enroll new voter
POST	/api/admin/toggle-status	Toggle election active
POST	/api/admin/reset	Reset all voter participation
GET	/api/admin/stats	Turnout statistics
POST	/api/admin/evidence	Create evidence document
DELETE	/api/admin/evidence/{docId}	Delete evidence document
POST	/api/admin/issues	Create campus issue
DELETE	/api/admin/issues/{issueId}	Delete campus issue
POST	/api/admin/seed-manifesto	Reseed demo data

B.6 Request/Response Examples

B.6.1 Login

```
POST /api/auth/login
{ "voterId": "VOTER001", "password": "voter123" }

Response 200:
{ "message": "Login successful", "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": { "voterId": "VOTER001", "role": "Voters" } }
```

Listing B.1: Login request/response

B.6.2 Vote Submission

```
POST /castVote
```

```
Authorization: Bearer <token>
{ "candidateId": "1" }

Response 200:
{ "message": "Vote recorded on Hedera!",
  "transactionId": "0.0.12345@1234567890.123456789",
  "status": "SUCCESS" }
```

Listing B.2: Vote submission request/response

B.6.3 Error Responses

```
401: { "error": "Access token required" }
403: { "error": "Invalid or expired token" }
403: { "error": "This voter has already voted." }
400: { "error": "Invalid candidate selected." }
502: { "error": "Ledger submission failed, vote not recorded" }
500: { "error": "Failed to broadcast vote to ledger." }
```

Listing B.3: Common error responses

Appendix C

Deployment Guide

C.1 Prerequisites

- Docker 24+ and Docker Compose 2+
- Node.js compatible with the project's installed dependencies
- MongoDB (or Docker)
- Hedera Testnet account with HBAR for fees
- HCS Topic ID (create via `createTopic.js`)

C.2 Environment Configuration

Create `backend/.env` from template:

```
PORT=5000
MONGO_URI=mongodb://mongodb:27017/evoting_db
JWT_SECRET=your-256-bit-secret-key-here
HEDERA_NETWORK=testnet
HEDERA_ACCOUNT_ID=0.0.<ACCOUNT_ID>
HEDERA_PRIVATE_KEY=302e020100300506032b657004220420...
HCS_TOPIC_ID=0.0.<TOPIC_ID>
BACKEND_URL=http://localhost:5000
```

Listing C.1: Backend .env template

Frontend config (`frontend/.env`):

```
VITE_BACKEND_URL=http://localhost:5000
```

Listing C.2: Frontend .env

C.3 Local Development with Docker Compose

Listing C.3: Start development stack

```
cd Blockchain-Based-Voting-System
docker compose up --build
```

Services:

- Frontend: `http://localhost:5173` (Vite dev server)
- Backend API: `http://localhost:5000`
- MongoDB: `localhost:27017`

C.4 Local Development Without Docker

Listing C.4: Manual startup

```
# Terminal 1: MongoDB
mongod --dbpath ./data/db

# Terminal 2: Backend
cd backend
npm ci
npm run dev  # or npm start

# Terminal 3: Frontend
cd frontend
npm ci
npm run dev
```

C.5 Illustrative Production Deployment (AWS)

C.5.1 Proposed Architecture

The following architecture is illustrative and has not been deployed or measured as part of the current prototype.

- **ECS Fargate:** Backend + Frontend containers
- **Application Load Balancer:** TLS termination, routing
- **CloudFront:** Global CDN for frontend static assets
- **MongoDB Atlas:** Managed replica set
- **Route 53:** DNS, custom domain
- **Secrets Manager:** JWT secret, Hedera keys
- **CloudWatch:** Logs, metrics, alarms

C.5.2 Key Production Changes

1. Set `NODE_ENV=production`
2. Use strong `JWT_SECRET` (256-bit, from Secrets Manager)
3. Enable `helmet.js`, `rate-limiter` in Express
4. Configure CORS for production domain only
5. Use MongoDB Atlas connection string with TLS
6. Switch Hedera to `mainnet` (after governance review)
7. Enable CloudFront caching for frontend build
8. Set up health checks: `GET /api/info`
9. Configure auto-scaling for ECS services

C.6 Hedera Mainnet Considerations

- **Governance Review:** Required for Mainnet deployment
- **Account Funding:** Mainnet HBAR for transaction fees
- **Topic Creation:** Use `Client.forMainnet()`, create topic
- **Mirror Nodes:** `https://mainnet.mirrornode.hedera.com`
- **HashScan:** `https://hashscan.io/mainnet`
- **Compliance:** KYC/AML for voter onboarding if required

C.7 Monitoring & Maintenance

- **Health Checks:** `/api/info` (200 OK), MongoDB ping
- **Logs:** Structured JSON to CloudWatch/ELK
- **Metrics:** Vote latency, error rates, turnout
- **Alerts:** HCS submission failures, DB connection drops
- **Backups:** MongoDB Atlas automated snapshots
- **Updates:** Monthly npm audit, dependency updates

Appendix D

User Manual

D.1 Voter Guide

D.1.1 Getting Started

1. Open the application URL in your browser
2. Click **Login** on the landing page
3. Enter your **Voter ID** and **Password** (provided by admin)
4. You will be redirected to the **Manifesto Dashboard**

D.1.2 Reviewing Candidates

1. On the **Candidates** tab, browse all candidates
2. Click any candidate card to expand their **Manifesto Promises**
3. Each promise shows: Category, Title, Current Problem, Proposed Solution, Expected Benefit, Timeline, Implementation Steps
4. Click **Evidence** links to view supporting documents
5. Use the **Compare** tab for side-by-side candidate comparison

D.1.3 Understanding Campus Issues

1. Navigate to the **Issues** tab
2. View priority issues with urgency badges (High/Medium/Low)
3. Each issue shows: Category, Description, Current Status, Proposed Solutions

D.1.4 Reviewing Evidence

1. Navigate to the **Evidence** tab
2. Browse verified documents (reports, letters, surveys)
3. Filter by candidate or category
4. Each document shows: Title, Type, Date, Issuing Authority, Verification Status, Key Findings

D.1.5 Casting Your Vote

1. After reviewing, click **Proceed to Vote** button
2. On the ballot page, select your preferred candidate
3. Click **Submit Vote**
4. Confirm in the dialog: *Are you sure you want to vote for [Candidate]?*
5. Wait for blockchain confirmation (3-5 seconds)
6. Success screen shows: **Transaction ID** and **HashScan Link**
7. Click the HashScan link to verify your vote on the public ledger

D.1.6 Important Notes

- **One vote only:** Once submitted, you cannot vote again
- **Verification:** Save your Transaction ID for future reference
- **Logout:** Use the logout button when done

D.2 Admin Guide

D.2.1 Initial Setup

1. Login with Admin credentials (default: ADMIN001 / admin123)
2. You will land on the **Admin Dashboard**
3. Verify election status and candidate list

D.2.2 Managing Candidates

1. Go to **Candidates** tab
2. **Add Candidate:** Click *Add Candidate*, fill Name, Party, optional Avatar URL
3. **Edit Profile:** Click *Edit* on any candidate to update details
4. **Manifesto Editor:** Click *Manifesto* to manage promises
    - Add promises with: Category, Title, Problem, Solution, Benefit, Timeline, Steps
    - Link evidence documents via Evidence Ref field
5. **Performance Records:** For incumbents, track promise fulfillment

D.2.3 Managing Voters

1. Go to **Voters** tab
2. **Enroll Voter:** Click *Enroll Voter*, enter Voter ID, Name
3. System generates secure password (displayed once - share securely)
4. **View Status:** See who has voted (hasVoted = true)

D.2.4 Managing Campus Issues

1. Go to **Issues** tab
2. **Create Issue:** Title, Category, Urgency, Description, Solutions
3. **Update/Delete:** Manage existing issues

D.2.5 Managing Evidence Documents

1. Go to **Evidence** tab
2. **Create Document:** Title, Type, Date, Authority, Summary, Verification Status
3. Link to candidate and category
4. Add key findings and file snippet for preview

D.2.6 Election Controls (Settings Tab)

1. **Toggle Election:** Open/close voting period
2. **Reset Participation:** Clear all hasVoted flags (use before new election)
3. **View Stats:** Real-time turnout percentage
4. **Reseed Data:** Restore demo manifesto data

D.3 Troubleshooting

Table D.1: Common Issues

Issue	Solution
"Invalid token"	Login again, token expired
"Already voted"	One vote per election, cannot revoke
"Voting closed"	Admin must toggle election active
"Candidate not found"	Refresh page, contact admin
Hedera error	Wait and retry, check testnet status

Appendix E

Test Cases and Results

E.1 Unit Tests

Table E.1: Backend Unit Test Coverage

Test File	**Tests**	**Coverage**
auth.test.js	5	Credential validation and JWT role claims
hederaErrors.test.js	2	Hedera error-message handling
Total	**7**	**7 passed in local run**

E.2 Integration Tests

Table E.2: Integration Test Scenarios

Scenario	**Steps**	**Expected**
Full vote flow	Login → Manifesto → Vote → Verify	HCS tx, receipt, tally
Admin workflow	Login → Add candidate → Enroll voter → Toggle	All operations succeed
Concurrent votes	10 simultaneous vote submissions	All processed, no duplicates
Token expiry	Wait for JWT exp → API call → Refresh	403 then new token

E.3 End-to-End Manual Test Checklist

Table E.3: Manual E2E Test Checklist

Test Case	Status	Notes
Admin login with default credentials	Not recorded	Requires environment-backed E2E run
Voter login with default credentials	Not recorded	Requires environment-backed E2E run
Invalid login rejected	Covered	`auth.test.js`
Expired token rejected	Not recorded	Requires API integration test
Manifesto dashboard loads all data	Not recorded	Requires MongoDB-backed run
Vote submission creates HCS tx	Not recorded	Requires funded Hedera testnet run
Duplicate vote rejected	Not recorded	Requires MongoDB-backed run
Mobile responsive layout	Not recorded	Requires browser test or screenshots
Logout clears session	Not recorded	Requires browser test

E.4 Load Test Results

Table E.4: Concurrent Vote Load Test (Hedera Testnet)

Concurrent Users	Success Rate	Avg Latency
5	100%	3.8s
10	100%	4.1s
25	96%	4.5s
50	92%	4.8s

Note: No load-test execution is included in the current repository. Any Hedera network throughput figure must be cited as provider capability rather than as a measurement of this application.

Appendix F

Configuration Files

F.1 Backend package.json

```
{
  "name": "evoting-backend",
  "version": "1.0.0",
  "description": "Blockchain-Based Voting System Backend",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "jest --coverage",
    "create-topic": "node createTopic.js",
    "seed": "node seedData.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^8.0.0",
    "jsonwebtoken": "^9.0.2",
    "bcryptjs": "^2.4.3",
    "@hashgraph/sdk": "^2.40.0",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "helmet": "^7.1.0",
    "express-rate-limit": "^7.1.5"
  },
  "devDependencies": {
    "nodemon": "^3.0.2",
```

```
    "jest": "^29.7.0"
  }
}
```

Listing F.1: backend/package.json

F.2 Frontend package.json

```
{
  "name": "evoting-frontend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "oxlint"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.21.0",
    "jwt-decode": "^4.0.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.1",
    "vite": "^5.0.10",
    "oxlint": "^0.2.0"
  }
}
```

Listing F.2: frontend/package.json

F.3 Docker Compose

```
version: '3.8'

services:
  mongodb:
```

```
    image: mongo:8
    container_name: evoting-mongodb
    volumes:
      - mongodb_data:/data/db
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_DATABASE: evoting_db

  backend:
    build: ./backend
    container_name: evoting-backend
    ports:
      - "5000:5000"
    environment:
      - MONGO_URI=mongodb://mongodb:27017/evoting_db
      - JWT_SECRET=${JWT_SECRET}
      - HEDERA_NETWORK=${HEDERA_NETWORK}
      - HEDERA_ACCOUNT_ID=${HEDERA_ACCOUNT_ID}
      - HEDERA_PRIVATE_KEY=${HEDERA_PRIVATE_KEY}
      - HCS_TOPIC_ID=${HCS_TOPIC_ID}
    depends_on:
      - mongodb
    env_file:
      - backend/.env

  frontend:
    build: ./frontend
    container_name: evoting-frontend
    ports:
      - "5173:80"
    depends_on:
      - backend

volumes:
  mongodb_data:
```

Listing F.3: docker-compose.yml

F.4 Vite Configuration

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true
      }
    }
  },
  build: {
    outDir: 'dist',
    sourcemap: true
  }
})
```

Listing F.4: frontend/vite.config.mjs

F.5 Nginx Configuration

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api {
        proxy_pass http://backend:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
```

```
        proxy_cache_bypass $http_upgrade;
    }
}
```

Listing F.5: frontend/nginx.conf

F.6 Backend Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

Listing F.6: backend/Dockerfile

F.7 Frontend Dockerfile

```
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing F.7: frontend/Dockerfile

Appendix G

Glossary

aBFT (Asynchronous Byzantine Fault Tolerance) A consensus property guaranteeing safety and liveness in asynchronous networks even when up to 1/3 of nodes are malicious.

Blockchain A distributed ledger technology where records (blocks) are cryptographically linked and immutable.

Client-Side Trust Security assumption that the user's browser/device executes code as intended, without malicious modification.

Consensus Timestamp The agreed-upon time assigned to a transaction by the network, ensuring fair ordering.

CORS (Cross-Origin Resource Sharing) Browser security mechanism controlling which origins can access resources.

DApp (Decentralized Application) An application that runs on a decentralized network rather than a single server.

DLT (Distributed Ledger Technology) The technological infrastructure underlying blockchain systems.

Gas Computational fee unit on Ethereum and similar platforms; not used in Hedera HCS.

HCS (Hedera Consensus Service) Hedera's topic-based messaging service providing fair ordering and consensus timestamps.

HBAR Native cryptocurrency of the Hedera network, used for transaction fees.

Hashgraph Hedera's consensus algorithm using virtual voting and gossip protocol for aBFT.

HCS Topic A named channel on Hedera for submitting ordered, timestamped messages.

JWT (JSON Web Token) Compact, URL-safe token format for transmitting claims between parties.

KYC (Know Your Customer) Identity verification process required by financial regulations.

Mainnet The production, live version of a blockchain network with real economic value.

Mirror Node Read-only Hedera node providing historical query access to consensus data.

Nonce A number used once, typically for preventing replay attacks.

Optimistic Locking Concurrency control strategy assuming conflicts are rare, checking at commit time.

Permissioned Ledger Blockchain where validator identities are known and approved (e.g., Hyperledger Fabric).

Proof of Stake (PoS) Consensus mechanism where validators stake tokens as collateral.

Proof of Work (PoW) Consensus mechanism requiring computational puzzle solving.

Receipt-freeness Voting property preventing voters from proving how they voted to a coercer.

Rollback Reverting a database change when a subsequent operation fails.

Smart Contract Self-executing code on a blockchain that enforces predefined rules.

Testnet A test version of a blockchain network with no real economic value.

TPS (Transactions Per Second) Measure of blockchain throughput capacity.

Virtual Voting Hashgraph's method of calculating consensus without actual voting messages.

Web2 Traditional centralized web architecture (servers, databases, client-server).

Web3 Decentralized web architecture using blockchain and peer-to-peer protocols.

Web2.5 Hybrid architecture combining Web2 identity/UX with Web3 consensus/verifiability.

ZK-Proof (Zero-Knowledge Proof) Cryptographic proof that a statement is true without revealing secret inputs.